

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Быстрые методы сортировки»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

I. Сортировка Ломута

1. Постановка задачи

Отсортировать массив методом Ломута

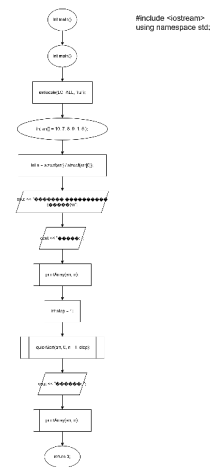
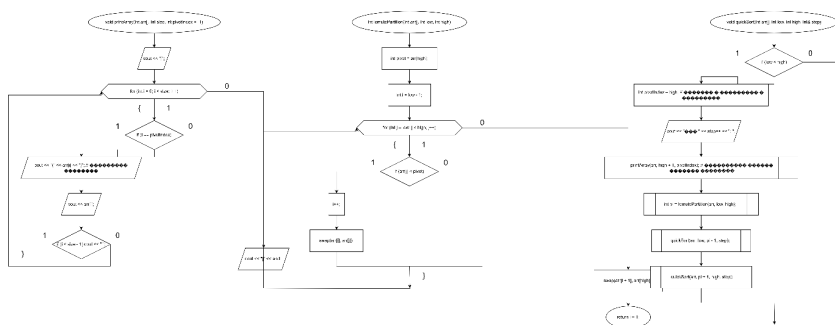
2. Анализ задачи

Шаг 1. Выбирается опорный элемент – pivot

Шаг 2. Все элементы меньше опорного располагаем слева от него, все что больше справа

Шаг 3. Рекурсивно повторяем эти действия, пока не получим отсортированный массив

3. Блок-схема



4. Код

```
#include <iostream>
using namespace std;
```

```
void printArray(int arr[], int size, int pivotIndex = -1) {
    cout << "[";
    for (int i = 0; i < size; i++) {
        if (i == pivotIndex)
            cout << "(" << arr[i] << ")"; // Подсветка опорного
        else
            cout << arr[i];
        if (i < size - 1) cout << " ";
    }
    cout << "]" << endl;
}
```

```
int lomutoPartition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
}
```

```

        return i + 1;
    }

void quickSort(int arr[], int low, int high, int& step) {
    if (low < high) {
        int pivotIndex = high; // Опорный — последний в диапазоне

        cout << "Шаг " << step++ << ": ";
        printArray(arr, high + 1, pivotIndex); // Показываем только текущий диапазон

        int pi = lomutoPartition(arr, low, high);

        quickSort(arr, low, pi - 1, step);
        quickSort(arr, pi + 1, high, step);
    }
}

int main() {
    setlocale(LC_ALL, "ru");
    int arr[] = { 10, 7, 8, 9, 1, 5 };
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Быстрая сортировка (Ломут)\n";
    cout << "Старт: ";
    printArray(arr, n);

    int step = 1;
    quickSort(arr, 0, n - 1, step);

    cout << "Готово: ";
    printArray(arr, n);
    return 0;
}

```

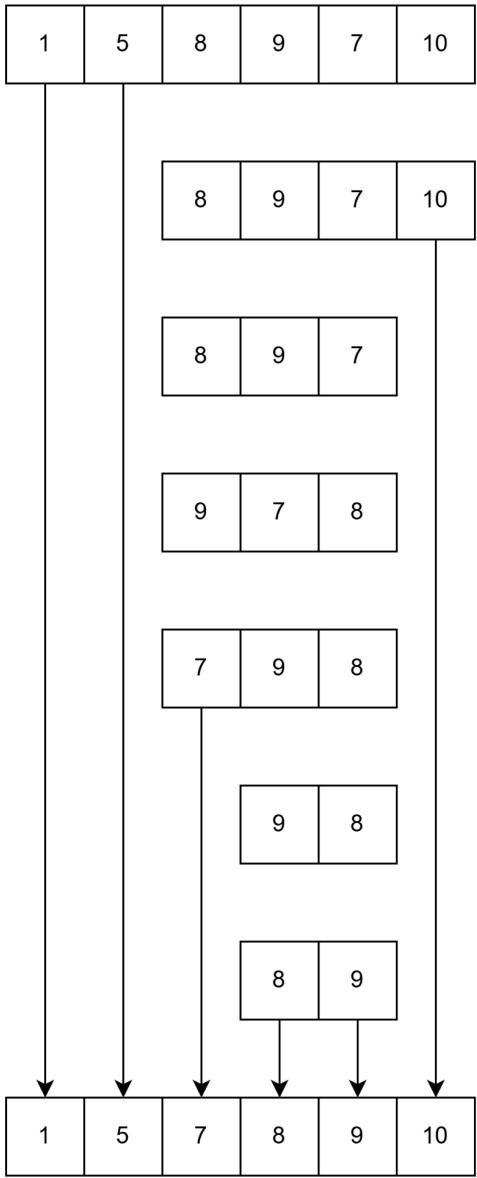
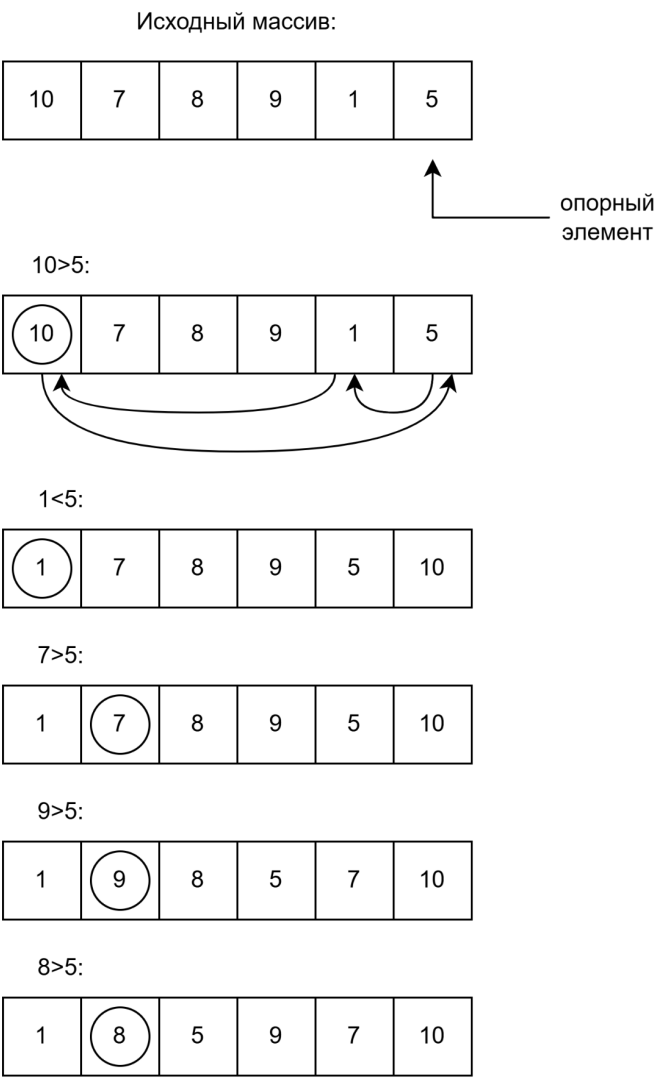
5. Результат выполнения программы

```

Быстрая сортировка (Ломут)
Старт: [10 7 8 9 1 5]
Шаг 1: [10 7 8 9 1 (5)]
Шаг 2: [1 5 8 9 10 (7)]
Шаг 3: [1 5 7 9 10 (8)]
Шаг 4: [1 5 7 8 10 (9)]
Готово: [1 5 7 8 9 10]

```

6. Графическое решение



II. Сортировка Хоара

1. Постановка задачи

Отсортировать массив методом Хоара

2. Анализ задачи

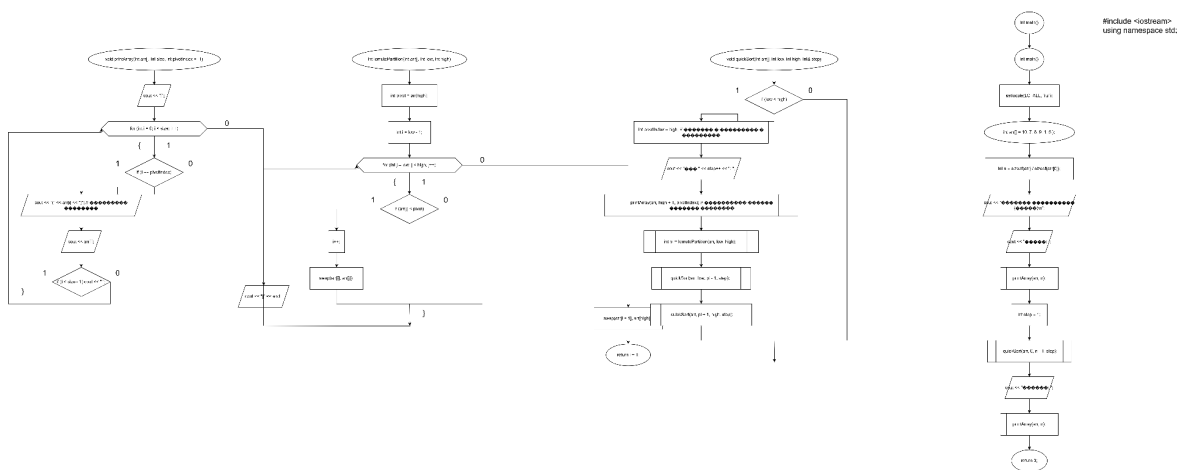
Шаг 1. Если диапазон менее 2-х элементов, прекращаем действие (массив уже отсортирован)

Шаг 2. За опорные элемент выбираем элемент в середине

Шаг 3. Разбиваем диапазон на части: меняем порядок таким образом, чтобы элементы меньше опорного элемента шли до него, а элементы больше – после него

Шаг 4. Рекурсивно применяем алгоритм к полученным частям

3. Блок-схема



4. Код

```
#include <iostream>
using namespace std;
```

```
int step = 0;
```

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

```
void printArray(int arr[], int size, const char* prefix = "") {
    cout << prefix;
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
void printStep(int arr[], int size, int depth, int pivot, int low, int high, int pl) {
    cout << "Шаг " << ++step << ": ";
    printArray(arr, size, "");
}
```

```

int partitionHoare(int arr[], int low, int high, int depth) {
    int pivot = arr[low + (high - low) / 2];
    int i = low - 1;
    int j = high + 1;

    while (true) {
        do { i++; } while (arr[i] < pivot);
        do { j--; } while (arr[j] > pivot);

        if (i >= j) {
            printStep(arr, (high - low + 1) + (low), depth, pivot, low, high, j);
            return j;
        }

        swap(arr[i], arr[j]);
    }
}

void quickSortHoare(int arr[], int low, int high, int depth, int totalSize) {
    if (low < high) {
        int pI = partitionHoare(arr, low, high, depth);

        quickSortHoare(arr, low, pI, depth + 1, totalSize);
        quickSortHoare(arr, pI + 1, high, depth + 1, totalSize);
    }
}

void quickSort(int arr[], int size) {
    quickSortHoare(arr, 0, size - 1, 0, size);
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 4, 9, 7, 6, 2, 3, 8 };
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Исходный массив: ";
    printArray(arr, n);

    quickSort(arr, n);

    cout << "Отсортированный массив: ";
    printArray(arr, n);

    return 0;
}

```

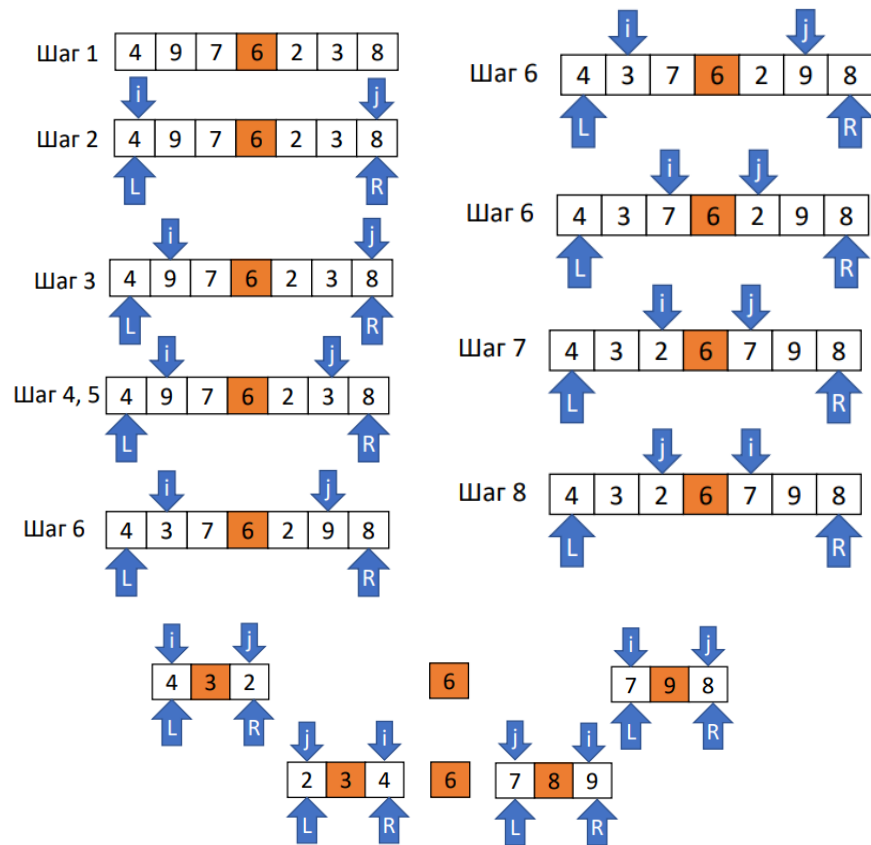
5. Результат выполнения программы

```

Исходный массив: 4 9 7 6 2 3 8
Шаг 1: 4 3 2 6 7 9 8
Шаг 2: 2 3 4 6
Шаг 3: 2 3
Шаг 4: 2 3 4 6
Шаг 5: 2 3 4 6 7 8 9
Шаг 6: 2 3 4 6 7 8
Отсортированный массив: 2 3 4 6 7 8 9

```

6. Графическое решение



III. Сортировка Шелла

1. Постановка задачи

Отсортировать массив методом Шелла

2. Анализ задачи

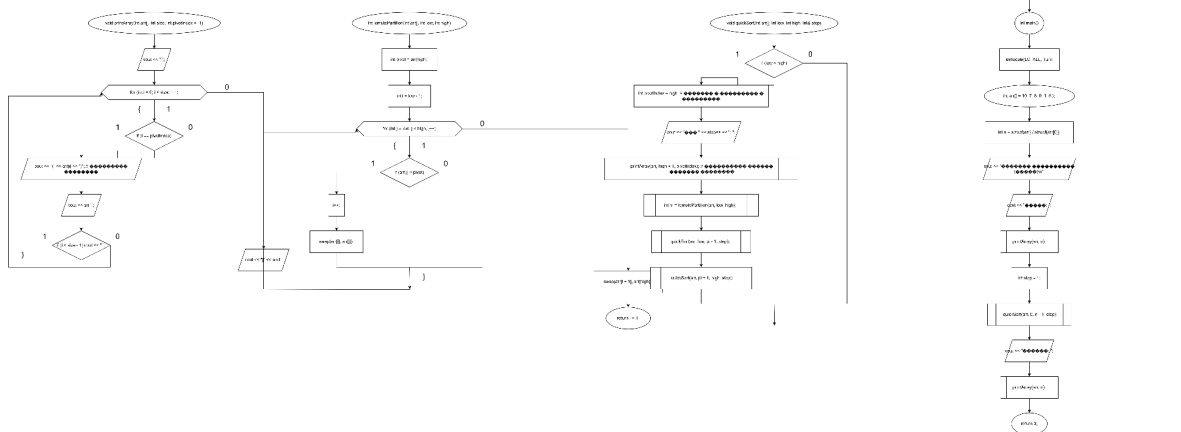
Шаг 1. Вычисляется расстояние между элементами – step (gap) - Начальный шаг обычно равен $n/2$, где n — размер массива. Затем шаг уменьшается (обычно делится на 2).

Шаг 2. Массив разбивается на списки элементов, отстающих друг от друга на step - Элементы с интервалом step образуют подпоследовательности для сортировки.

Шаг 3. Элементы каждого списка сортируются методом вставки - Применяется сортировка вставками к каждому подмножеству элементов.

Шаг 4. Происходит объединение списков обратно в массив, уменьшается step - Если $step > 0$, то происходит переход обратно на шаг 2. Когда $step = 1$, выполняется финальная сортировка вставками.

3. Блок-схема



4. Код

```
#include <iostream>
using namespace std;
```

```
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

```
void shellSort(int arr[], int size) {
    for (int step = size / 2; step > 0; step /= 2) {
        cout << "\nСортировка с шагом " << step << endl;

        for (int i = step; i < size; i++) {
            int temp = arr[i];
            int j;
```



```

        for (j = i; j >= step && arr[j - step] > temp; j -= step) {
            arr[j] = arr[j - step];
        }

        arr[j] = temp;

        printArray(arr, size);
    }
}

int main() {
    setlocale(LC_ALL, "ru");

    int arr[] = { 5, 2, 9, 1, 7, 3, 4, 5, 2 };
    int size = sizeof(arr) / sizeof(arr[0]);

    cout << "Исходный массив: ";
    printArray(arr, size);

    shellSort(arr, size);

    cout << "\nОтсортированный массив: ";
    printArray(arr, size);

    return 0;
}

```

5. Результат выполнения программы

```

Исходный массив: 5 2 9 1 7 3 4 5 2

Сортировка с шагом 4
5 2 9 1 7 3 4 5 2
5 2 9 1 7 3 4 5 2
5 2 4 1 7 3 9 5 2
5 2 4 1 7 3 9 5 2
2 2 4 1 5 3 9 5 7

Сортировка с шагом 2
2 2 4 1 5 3 9 5 7
2 1 4 2 5 3 9 5 7
2 1 4 2 5 3 9 5 7
2 1 4 2 5 3 9 5 7
2 1 4 2 5 3 9 5 7
2 1 4 2 5 3 9 5 7
2 1 4 2 5 3 7 5 9

Сортировка с шагом 1
1 2 4 2 5 3 7 5 9
1 2 4 2 5 3 7 5 9
1 2 2 4 5 3 7 5 9
1 2 2 4 5 3 7 5 9
1 2 2 3 4 5 7 5 9
1 2 2 3 4 5 7 5 9
1 2 2 3 4 5 5 7 9
1 2 2 3 4 5 5 7 9

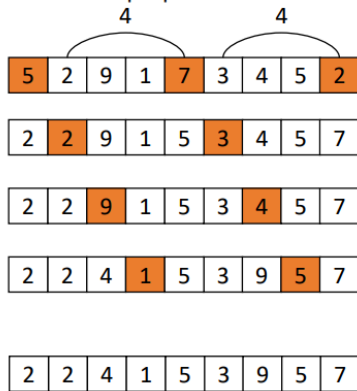
Отсортированный массив: 1 2 2 3 4 5 5 7 9

```

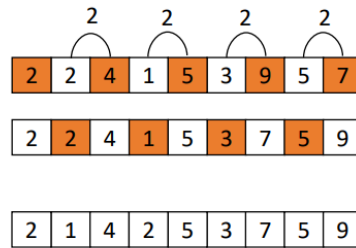
6. Графическое решение

Шаг = размер массива / 2 = 9 / 2 = 4

1 этап – сортировка с шагом 4



2 этап – сортировка с шагом 2



3 этап – сортировка с шагом 1

